

星云知识助手检索系统与 Qdrant 运行手册

文档编号: XY-RAG-RET-003

版本: 1.0

发布日期: 2026-06-23

维护团队: 智能平台组

1. 系统结构

查询流程由文本向量化、候选检索、过滤、可选重排序和结果返回组成。生成模型不直接访问全部文档，而只接收检索出的少量上下文。检索阶段的错误会限制生成阶段，因此必须单独评测检索质量。

当前基线使用 BAAI/bge-small-zh-v1.5 生成稠密向量，Qdrant 保存文档块、向量和元数据。查询向量与文档向量采用余弦相似度。基线 Top K 为 3，线上问答可设置分数阈值，离线 Recall@K 评测不使用阈值。

2. 稠密检索

稠密检索把查询和文档映射到同一向量空间，适合处理同义表达和自然语言改写。例如“机器不在线”和“设备离线”可能在向量空间中相近，即使没有完全相同的关键词。

稠密检索的弱点是精确代码、罕见型号和短字符串。例如 E-204 与 E-240 只差字符顺序，但业务含义可能完全不同。对这类查询，应结合关键词检索或元数据过滤。

3. 关键词检索

关键词检索强调词项匹配。BM25 会考虑词频、文档频率和文档长度，在查找产品编号、故障码、人名、条款编号时通常更稳定。

关键词检索并不理解所有语义改写。用户问“设备突然断开网络”时，文档若只写“离线”，纯关键词方法可能漏检。因此系统应通过实验比较稠密、关键词和混合方法，而不是预先断言某一种始终最好。

4. 混合检索

混合检索同时获取稠密候选和稀疏候选，再进行融合。Qdrant 支持在同一个点中保存多种向量表示，并通过混合查询组合结果。常用融合方法包括 Reciprocal

Rank Fusion。融合前应保证各检索器的候选数量足够，否则主查询可能没有可用结果。

推荐先分别记录 Dense 和 BM25

的原始排名，再记录融合排名。这样在结果异常时，可以判断问题来自哪条检索路径。

5. 重排序

重排序模型只处理初步召回的少量候选，以更细致的查询-文档相关性信号重新排序。它通常提高前几名精度，但会增加延迟和计算成本。

基线实验建议先取二十个候选，再重排到前五名。若初始召回没有包含正确块，重排序无法创造缺失证据。因此必须同时观察 Recall@20 和重排后的 MRR 或 Precision@5。

6. Qdrant 集合设计

建议集合名称包含环境和版本，例如 xingyun_kb_dev_v1。向量维度必须与 Embedding 模型输出一致。更换模型时若维度或向量含义改变，应建立新集合，不得把新旧向量混写。

每个 point 保存一条文档块。payload 包含

source、chunk_id、document_id、version、status、department、access_level、customer_scope 和 effective_date。常用过滤字段应建立 payload 索引。

7. 过滤规则

查询时至少添加 status=effective。随后根据用户角色限制 access_level 和 department。客户专属资料还需要 customer_scope 过滤。

过滤不参与相关性打分，它的作用是缩小合法候选范围。过滤过严会造成零结果，过滤过松会造成越权。出现零结果时，应记录是哪一个条件导致候选为空。

示例：普通客服查询产品升级，应只访问公开产品资料和客服制度；售后工程师可额外访问技术手册；主管查看事件复盘时还需部门和密级匹配。

8. Top K 与分数阈值

Top K 决定返回候选数量。K 太小可能漏掉正确证据，K 太大则增加噪声和生成成本。阈值用于丢弃相似度过低的结果，但不同模型、数据集和距离度量的分数不可直接套用。

线上可使用 `top_k=5`、`score_threshold=0.45` 作为初始配置，但该值只是实验起点。离线比较 `Recall@3`、`Recall@5` 和 `Recall@10` 时，应关闭阈值，确保每个查询返回固定数量候选。

9. 查询预处理

查询应去除首尾空格，保留故障码、版本号和否定词。不要把所有文本强制转为小写后再丢失大小写敏感标识。对于“不要重启”和“可以重启”，否定词决定业务含义，任何改写都必须谨慎。

可以从查询中提取结构化字段，例如设备型号 `XN-500`、版本 `3.2.1`、客户范围 `A` 类。提取结果只能用于辅助过滤，低置信度字段不应强制过滤。

10. 索引写入

入库流程先生成文档块，再批量生成向量，最后写入 `Qdrant`。写入完成后要核对预期块数与实际 `point` 数。若重复执行同一文件，应使用稳定的文档和块标识实现覆盖或跳过，避免重复数据。

批量任务应记录嵌入模型、模型版本、向量维度、批大小、开始时间、结束时间和失败数量。只记录“上传成功”不足以支持复现。

11. 常见故障

11.1 集合不存在

检查 `Qdrant` 服务是否启动、连接地址是否正确、集合名称是否匹配。应用不应在连接失败时静默创建错误名称的新集合。

11.2 向量维度不匹配

通常发生在更换 `Embedding` 模型后仍写入旧集合。处理方式是确认模型维度，并新建集合重新索引。

11.3 检索结果为空

依次检查集合 `point` 数、查询向量、过滤条件和分数阈值。不要第一时间降低阈值，因为真正原因可能是权限过滤或数据未入库。

11.4 结果重复

检查文档是否重复上传、`chunk_overlap` 是否过大、`point ID` 是否稳定。可在返回结果中按 `document_id` 和 `chunk_id` 去重，但根本问题仍应在入库阶段解决。

11.5 相似但错误

查看正确块是否出现在更大的 K 中。如果在 Top 20 但不在 Top 5, 可以尝试重排序。如果 Top 20 也没有, 优先检查分块、查询表达和检索模型。

12. 监控指标

线上至少监控请求量、检索耗时、零结果率、平均返回块数、过滤后候选数和错误率。实验环境还要记录 Recall@K、MRR、命中位置分布和不同类型问题的表现。

仅看平均值可能掩盖问题。应分别统计故障码类、制度类、解释类和多文档类查询。

13. 备份与维护

集合变更前创建快照。模型升级、字段变更和大规模重分块都应使用新集合进行灰度验证。确认评测和权限测试通过后, 再切换别名或配置。

14. 安全注意事项

Qdrant 连接信息和 API Key 通过环境变量配置, 不写入代码仓库。生产服务限制网络访问来源。调试日志不得打印完整令牌或未经脱敏的客户文档正文。

15. 参考依据

本文为原创教学语料。混合检索、过滤和多阶段查询部分参考 Qdrant 官方文档; 检索器与管线连接方式参考 Haystack 官方文档。